chat mlp evo

> Archivio di una conversazione ChatGPT condivisa pubblicamente. Sorgente: https://chatgpt.com/share/6ad1c0d5-49ec-83eb-a79b-85b04b938835 Messaggi visibili: 42. Le immagini e i file pubblici recuperati sono locali; i contenuti non accessibili sono segnalati.

USER

puoi leggere o controllare quello che ho fatto in una sessione work?

ASSISTANT

Sì, **posso controllare il lavoro svolto in una sessione ChatGPT Work**, ma con una distinzione importante.

Non posso aprire autonomamente un'altra sessione Work e leggerne direttamente la conversazione o la cronologia delle operazioni. Posso però verificare i risultati del lavoro attraverso le risorse a cui ho accesso.

Nel caso di RumiAI, per esempio, posso:

- **GitHub:** controllare commit, modifiche, branch, file e test prodotti dalla sessione Work.
- **Documentazione:** verificare i documenti creati o modificati.
- **Codice:** confrontare l'implementazione con le specifiche correnti di `rumiai-dev`.
- **Sessione Work:** analizzare un resoconto o una trascrizione che mi condividi.

La cosa interessante

Se Work ha lavorato sui repository GitHub di RumiAI, **posso effettuare una revisione indipendente del risultato**, senza bisogno di leggere tutta la conversazione Work.

Posso anche identificare eventuali errori, modifiche incomplete, incoerenze architetturali e test mancanti.

Ti chiedo una cosa: vuoi che controlli *cosa ha fatto* Work (comprese le decisioni e i ragionamenti), oppure vuoi che verifichi *il risultato effettivo* del suo lavoro su RumiAI?

Sono due verifiche differenti, e la seconda posso svolgerla direttamente tramite GitHub.

USER

voglio tutte e due le cose

ASSISTANT

Perfetto. Facciamo una **revisione completa e indipendente della sessione Work**, su due livelli.

Ho già effettuato un primo accesso a GitHub e verificato gli HEAD remoti correnti dei principali repository RumiAI. Ho anche letto le regole correnti di `rumiai-dev`, comprese quelle sulla coerenza tra specifiche, implementazione

e test.

---

1. Revisione della sessione Work

Voglio ricostruire:

- Quali istruzioni hai dato a Work.
- Quali decisioni ha preso autonomamente.
- Quali strumenti ha utilizzato e quali operazioni ha realmente eseguito.
- Quali problemi ha incontrato e come li ha risolti.
- Se ha rispettato le tue istruzioni e le regole RumiAI.
- Se ci sono errori, omissioni o affermazioni non dimostrate.

---

2. Verifica indipendente del risultato

Attraverso GitHub posso verificare direttamente codice, documentazione, commit, test e workflow, confrontandoli con le specifiche canoniche correnti.

Non mi limiterò quindi a fidarmi di quello che Work dichiara di aver fatto.

---

3. Primo controllo GitHub

<text color="secondary" size="xs">HEAD remoti verificati · 8 ottobre 2026</text>

Repository	HEAD
`rumiai-dev`	`017355db`
`rumiai-os`	`6a964ba3`
`rumiai-tests`	`89cd8308`
`pkg-catalog`	`7d638061`
`rumiai-web-control`	`8ed3ab88`
`rumiai-dev-PoCs`	`3c00148a`

Questo è il punto di partenza verificato, non ancora un giudizio sulla qualità del lavoro.

Mi manca soltanto identificare la sessione

Ho individuato riferimenti a diverse attività RumiAI, ma non posso stabilire con certezza quale sessione Work intendi.

Mandami il link alla sessione Work che vuoi controllare.

Se il link non è accessibile direttamente, puoi condividere la trascrizione o il resoconto della sessione. Potrò così confrontare le decisioni documentate con le modifiche effettivamente presenti su GitHub.

Il risultato sarà un unico rapporto con **decisioni corrette, errori, discrepanze, test verificati e interventi eventualmente necessari**, distinguendo sempre le evidenze dirette da ciò che non è stato possibile verificare.

USER

https://chatgpt.com/g/g-p-6a6aeaf208e081918b838f1936186ea8-rumiai/c/6ac6b1b8-acac-83eb-a25c-63064c034d9d

---

ASSISTANT

Ho provato ad accedere alla sessione Work tramite il link che mi hai fornito, ma **non riesco a leggerne direttamente il contenuto**: l'accesso a quella conversazione è bloccato.

Ho anche cercato di recuperarla attraverso il contesto delle conversazioni precedenti, ma non ho ottenuto una trascrizione identificabile con certezza come quella del link.

Posso comunque effettuare entrambe le verifiche che mi hai chiesto:

Verifica	**Stato**
Lettura delle istruzioni date a Work	<badge color="warning">Serve la trascrizione</badge>
Analisi delle decisioni e delle operazioni	<badge color="warning">Serve la trascrizione</badge>
Controllo indipendente dei repository GitHub	<badge color="success">Accessibile</badge>
Confronto con le specifiche RumiAI	<badge color="success">Accessibile</badge>
Verifica di implementazione e test	<badge>Da eseguire</badge>

Per completare la prima parte, apri la sessione Work e condividi qui il suo contenuto, preferibilmente tramite un'esportazione della conversazione o copiando il resoconto completo delle operazioni.

Una volta disponibile, potrò ricostruire il lavoro e confrontarlo con le evidenze effettive su GitHub, senza attribuire a Work modifiche che non posso dimostrare.

Non ho modificato alcun repository.

USER

ok farò copia ed incolla della conversazione in vari blocchi poi quando avrò finito, ti scriverò semplicemente "trascrizione completata", è tutto chiaro?

ASSISTANT

Chiarissimo.

Incolla pure la conversazione in tutti i blocchi necessari. **Non analizzerò, commenterò né eseguirò verifiche** durante l'invio.

Attenderò il messaggio esatto **«trascrizione completata»** prima di iniziare la revisione completa della sessione Work e il confronto con GitHub.

Per ogni blocco risponderò soltanto **«Ricevuto»**.

USER

vorrei costruire in casa ad un costo contenuto dell'hardware in silicio, ottico, fibre ottiche, o qualsiasi altra cosa ti venga in mente. questo hardware dovrebbe servire per far girare una ai, quindi diciamo calcolo parallelo, memoria distribuita, alu, o altro che ti viene in mente, magari combinato con sistemi caotici e sincronizzazione, reservoir computing ed altro che ritieni più pertinente. l'obiettivo è costruire in casa un prototipo che dimostri un modello evoluto ed innovativo che incrementi le prestazioni e le capacità delle ai attuali sfruttando modelli completamente nuovi ispirate dalle matematiche più avanzate e da architetture hardware che ne esprimano le massime capacità.

ASSISTANT

Ricevuto

USER

ASSISTANT

Ricevuto

USER

che ne pensi di usare simulatori o crearne di appropriati per simulare delle idee per capire se vale la pena realizzarle?

ASSISTANT

Ricevuto

USER

Sì. Per questo progetto farei della simulazione il primo vero PoC, prima della scheda fisica che proponevo. Ci permetterebbe di esplorare molte architetture, capire quali proprietà producono un vantaggio e spendere sull’hardware solo quando abbiamo una domanda sperimentale precisa.

Un computer tradizionale può simulare un’architettura molto diversa dalla propria. Naturalmente, **il tempo impiegato dal simulatore non rappresenta la velocità del futuro dispositivo**: possiamo verificare il comportamento del modello, mentre prestazioni ed energia dell’hardware richiedono una valutazione separata.

Procederei con livelli di dettaglio crescenti:

Livello	Domanda da risolvere	Cosa simulare
Modello matematico	L'idea offre una capacità utile?	Dinamiche, memoria, connessioni, apprendimento
Vincoli realistici	Il vantaggio sopravvive alle imperfezioni?	Rumore, precisione limitata, ritardi, saturazione, variazioni dei componenti
Realizzazione fisica	Possiamo costruirla con componenti accessibili?	Circuito elettronico o componenti ottici concreti
Piccolo prototipo	Quanto è fedele la simulazione?	Misure su poche celle per correggere il modello

Userei simulatori esistenti, costruendo sopra di essi un piccolo ambiente dedicato ai nostri esperimenti.

Per le equazioni dinamiche potremmo usare solutori consolidati come **SUNDIALS**, che comprende strumenti per equazioni differenziali ordinarie e differenziali-algebriche. I modelli con ritardi o termini stocastici richiederebbero metodi appropriati al caso. Documentation for SUNDIALS documentation

Per l'elettronica sceglierei **ngspice**, eventualmente attraverso KiCad. È controllabile da un programma esterno: possiamo generare circuiti, cambiare parametri, lanciare simulazioni e raccogliere risultati automaticamente. Inoltre, **XSPICE consente di introdurre modelli comportamentali personalizzati**: utile quando vogliamo sperimentare una cella innovativa prima di definirne ogni transistor. Shared library

Per l'ottica partirei da modelli dei singoli componenti e della propagazione. Userei **Meep**, che simula i campi elettromagnetici, soltanto dove interferenza, geometria o materiali determinano il risultato: una simulazione dettagliata dell'intero sistema potrebbe diventare inutilmente costosa. MEEP Documentation

La parte da creare noi avrebbe poche responsabilità:

- descrivere celle, connessioni e parametri;
- applicare gli stessi ingressi alle architetture confrontate;
- addestrare e valutare ciascun modello;
- esplorare automaticamente combinazioni di parametri;
- conservare configurazioni e risultati riproducibili.

Il valore specifico sarebbe **trasformare un'idea in un esperimento confrontabile**. Inizialmente basta un programma piccolo; l'interfaccia grafica può arrivare quando sappiamo quali grandezze serve osservare.

Per il nostro primo esperimento confronterei:

1. un predittore semplice che utilizza campioni passati;
2. un reservoir convenzionale;
3. una rete con celle dai tempi di risposta differenti;
4. la stessa rete con accoppiamenti regolabili.

Il compito potrebbe essere **ricostruire grandezze non osservate di un sistema dinamico a partire da un segnale parziale e rumoroso**. Poi verificheremmo cosa accade cambiando velocità del segnale, rumore e disponibilità delle osservazioni.

Questo permetterebbe di rispondere a domande precise: servono davvero le diverse scale temporali? L'accoppiamento aggiunge informazione? La sincronizzazione aiuta oppure rende le celle ridondanti? Quanto costa adattare il modello?

Con il caos, anche il simulatore deve essere verificato. Un passo temporale troppo grande può alterare la dinamica; precisione irrealistica o componenti perfetti possono creare vantaggi che spariscono sul banco. Controllerei quindi la stabilità delle conclusioni variando passo, tolleranze, condizioni iniziali e imperfezioni. Su tempi lunghi, per sistemi caotici, confronterei anche proprietà statistiche e dinamiche, senza pretendere traiettorie identiche punto per punto.

Passerei alla costruzione quando il vantaggio:

- compare su dati tenuti fuori dall'addestramento;
- resiste a variazioni realistiche;
- supera confronti semplici con risorse comparabili;
- corrisponde a una realizzazione fisica plausibile.

Il prossimo passo che sceglierei è dunque un micro-PoC numerico con un solo compito e poche architetture confrontabili. Ho aggiornato il filone di ricerca con questa proposta; non ho ancora eseguito simulazioni.

ASSISTANT

Ricevuto

USER

voglio dirti anche una delle idee che vorrei valutare è il calcolo analogico. le ai dovrebbero simulare l'intelligenza del cervello che è puramente analogico e la natura dimostra che un ammasso di neuroni (analogici) di circa 20cm x 20cm può far girare un'intelligenza, magari non velocissima, ma molto potente. quindi la domanda è: riuscire ad usare delle tecnologie estremamente più veloci della trasmissione chimico-biologica dei neuroni, per riprodurre un qualcosa di simile al cervello umano.

ASSISTANT

Ricevuto

USER

Sì: costruire reti fisiche ispirate ai neuroni, con dinamiche molto più rapide di quelle biologiche, è possibile ed è già stato dimostrato. Considererei il calcolo analogico una delle direzioni principali da valutare per il tuo progetto. La questione decisiva è come combinare velocità, memoria e apprendimento in una struttura capace di sfruttarli.

Preciserei però "puramente analogico": il cervello combina dinamiche continue — tensioni, correnti ioniche, concentrazioni chimiche — con impulsi, gli *spike*, il cui momento di emissione può trasportare informazione. Gli spike stessi sono fenomeni fisici continui, non semplici bit. Esistono anche evidenze sperimentali di trasmissione che combina segnali graduati e potenziali d'azione. Quindi l'ispirazione biologica suggerisce di esplorare sia circuiti continui sia sistemi a eventi. pubmed.ncbi.nlm.nih.gov

Un esempio concreto è **BrainScaleS-2**, sviluppato a Heidelberg: integra 512 circuiti neuronali e circa 131.000 sinapsi, con un nucleo analogico e componenti digitali per controllo, comunicazione e apprendimento. Emula determinate dinamiche neurali circa **1.000 volte più velocemente del tempo biologico**. Quel fattore descrive la

velocità delle dinamiche emulate; ottenere capacità paragonabili a quelle umane rimane un problema distinto. www.kip.uni-heidelberg.de

La parte interessante dell'analogico è che **la fisica del circuito può eseguire direttamente una parte del calcolo**. Un condensatore accumula carica; correnti convergenti si sommano; transistor introducono non linearità. Queste proprietà possono realizzare integrazione, memoria temporanea e trasformazioni dei segnali. È un principio centrale dell'ingegneria neuromorfica, presente già nel lavoro di Carver Mead. authors.library.caltech.edu

Per il nostro esperimento, tradurrei l'ispirazione cerebrale in proprietà precise:

Proprietà da esplorare	**Possibile realizzazione**
Stato interno di un neurone	Tensione su un condensatore
Risposta non lineare e soglia	Circuito a transistor o comparatore
Connessioni eccitatorie e inibitorie	Accoppiamenti con intensità e segno regolabili
Memoria di durate diverse	Circuiti con costanti di tempo differenti
Apprendimento	Modifica dei collegamenti in funzione dell'attività e di un segnale di errore o ricompensa

Le prime righe sono relativamente accessibili. **Le connessioni modificabili e il modo in cui apprendono rappresentano una parte molto più impegnativa del progetto.** Anche BrainScaleS combina dinamiche analogiche con elaborazione digitale della plasticità, cioè dei cambiamenti delle connessioni. www.kip.uni-heidelberg.de

Il tuo riferimento alla compattezza del cervello è significativo: dimostra che un sistema fisico compatto può sostenere un'intelligenza molto potente. Tuttavia, quella capacità dipende anche dall'organizzazione delle connessioni, dagli stati interni e dall'apprendimento. Costruire elementi veloci ci fornisce un substrato; dobbiamo ancora individuare un'organizzazione efficace.

C'è poi un dettaglio importante: **accelerare tutto può accorciare anche la memoria**. In un modello ideale, se comprimiamo coerentemente tutti i tempi di un fattore 1.000, una traccia che durava un secondo durerà un millisecondo. Otteniamo la stessa evoluzione accelerata, a condizione di accelerare anche gli ingressi e gli altri processi. Ma un ambiente reale continua a cambiare alla propria velocità.

Per questo esplorerei una macchina con **elaborazione rapida e stati di memoria su più scale temporali**: alcuni velocissimi, altri persistenti, altri modificabili lentamente attraverso l'esperienza. L'accelerazione diventerebbe così una risorsa da distribuire.

La mia proposta concreta è partire dal simulatore, con **32–64 celle**, confrontando:

- una rete a stati continui e una rete a impulsi;
- collegamenti fissi e collegamenti che apprendono;
- memoria con una sola durata e memoria con durate diverse;
- comportamento ideale e comportamento con rumore, imprecisioni e ritardi realistici.

Come primo compito sceglierei **prevedere un segnale osservato parzialmente e adattarsi quando cambia il processo che lo genera**. Misureremmo errore, quantità di esperienza necessaria per adattarsi e conservazione di ciò che era stato appreso, confrontando il risultato con modelli digitali semplici.

Caos e sincronizzazione entrerebbero come parametri sperimentali: cercheremmo quali regimi migliorano queste prestazioni. Un'attività complessa o apparentemente "viva" da sola non dimostrerebbe una capacità utile.

Il candidato che valuterei per primo è quindi una rete elettronica analogica con memoria locale, connessioni modificabili e più scale temporali, assistita da un controllore per configurazione e misure. Un vantaggio misurabile in questo piccolo sistema sarebbe già una base concreta per decidere cosa costruire e come ampliarlo.

Ho registrato questa direzione nelle note di ricerca. Per ora abbiamo una proposta sperimentale fondata sulla letteratura; le simulazioni devono ancora essere eseguite.

ASSISTANT

Ricevuto

USER

puoi creare un poc da fa girare su una vm linux di github, o la tua? utilizzando simulatori e librerie open source. obiettivo simulare tramite componenti elettrici, elettronici, ottici e fibre ottiche dei comportamenti utili per le ai.

ASSISTANT

Ricevuto

USER

Verifico i repository e le istruzioni correnti di RumiAI, poi preparo un PoC eseguibile su Linux con simulatori open source e un confronto misurabile su un compito utile all'AI. Ran 7 commands e used GitHub Il PoC appartiene a `rumiai-dev-PoCs`; il filone di ricerca esiste già. Sto controllando anche le modifiche locali presenti, per preservarle. Imposterò il confronto distinguendo i risultati del simulatore da eventuali vantaggi del futuro hardware. Ran 8 commands, explored files e searched the web Preparo tre esperimenti sullo stesso compito temporale: una rete R-C lineare, una rete elettronica con non linearità e un anello optoelettronico con ritardo in fibra. Li confronterò con predittori digitali semplici. Per l'elettronica userò ngspice; per l'ottica un modello dei componenti, dichiarandone le approssimazioni. Ran 5 commands, explored files e edited 1 file La prima esecuzione funziona già nella VM Linux. È emerso un risultato utile: queste configurazioni fisiche non superano i predittori digitali sul compito temporale scelto, e la lettura della rete R-C è molto sensibile alle perturbazioni. Ora verifico le traiettorie elettroniche anche con ngspice su GitHub, per controllare l'accordo tra equazioni e circuito. Ran 6 commands e edited 1 file Verificando stato e remoto Git Il controllo automatico ha bloccato il push diretto su `main`, perché richiede un'autorizzazione esplicita alla pubblicazione sul ramo principale. Procedo con un ramo sperimentale separato, così posso eseguire la verifica richiesta su GitHub mantenendo `main` intatto. Ran 4 commands e used GitHub La prova completa nella mia VM ha confrontato otto configurazioni su tre semi casuali. Le celle a diodi riescono a rappresentare una trasformazione non lineare che la rete R-C lineare non cattura; sul compito con memoria, però, prevalgono ancora i predittori digitali. Il ramo sperimentale è pubblicato e GitHub sta eseguendo ngspice. Ran 3 commands, used GitHub, edited 2 files e explored files Anche la prova su GitHub è riuscita. Ngspice e il modello a equazioni concordano entro circa 41 microvolt nei controlli elettronici, e producono risultati molto vicini sui compiti AI. Sto salvando le evidenze e le istruzioni: il PoC è riutilizzabile, ma i risultati non giustificano ancora la costruzione di un acceleratore.

ASSISTANT

Ricevuto

USER

trascrizione completata

ASSISTANT

Revisione della sessione Work — RumiAI

<badge color="success"><icon name="check-circle" size="xs" inline/> PoC eseguito e verificato su GitHub</badge>

Ho confrontato la trascrizione con il codice pubblicato, il workflow GitHub Actions, i log dell'esecuzione, i risultati numerici e le regole correnti di RumiAI.

Il risultato principale è positivo: Work ha effettivamente costruito un simulatore sperimentale funzionante, lo ha eseguito su Linux e ha ottenuto risultati scientificamente utili, anche se non dimostrano ancora un miglioramento rispetto ai metodi digitali.

Ho però individuato alcune limitazioni sperimentali e un problema di sincronizzazione della documentazione RumiAI che meritano attenzione.

---

1. Che cosa ha realizzato Work

<box gap={2}> <row align="center" justify="between"> **<Link url="https://github.com/massimilianonardi-ai/rumiai-dev-PoCs/pull/1" title="Apri il PoC 058"/>** <badge>Draft PR</badge> </row> <text color="secondary" size="sm">Repository `rumiai-dev-PoCs` · Ramo `poc/058-physical-reservoir-simulation`</text> <divider color="subtle"/> <grid columns={2} gap={3}> <grid-item> <text color="secondary" weight="medium" size="xs">Simulazioni confrontate</text> <title size="xl" color="default" tabularNums>8</title> </grid-item> <grid-item> <text color="secondary" weight="medium" size="xs">Semi casuali</text> <title size="xl" color="default" tabularNums>3</title> </grid-item> <grid-item> <text color="secondary" weight="medium" size="xs">Esito GitHub Actions</text> <row align="center" gap={1}> <icon name="check-circle" color="success"/> **Success** </row> </grid-item> <grid-item> <text color="secondary" weight="medium" size="xs">Simulazione hardware fisico</text> **Non ancora costruito** </grid-item> </grid> </box>

Il prototipo comprende tre famiglie di modelli:

- **Elettronica analogica:** 16 celle R-C, varianti con diodi non lineari e accoppiamenti resistivi.
- **Optoelettronica:** modulatore ottico, laser, rivelatore e retroazione attraverso un modello di fibra ottica.
- **Confronti digitali:** regressori temporali e una rete *Echo State Network*.

L'apprendimento è attualmente digitale e riguarda soltanto lo stadio di lettura degli stati.

Ho verificato anche che Work ha mantenuto il codice su un ramo sperimentale, senza modificarne il `main` e senza integrare prematuramente questi meccanismi in `rumiai-os`.

2. Verifica dell'esecuzione effettiva

<box gap={2}> <row align="center" justify="between"> **<Link url="https://github.com/massimilianonardi-ai/rumiai-dev-PoCs/actions/runs/37754200236" title="GitHub Actions — Run 37754200236"/>** <badge color="success">Success</badge> </row> <text color="secondary" size="sm">8 ottobre 2026 · Ubuntu 24.04 · ngspice · Python 3.12</text> <list marker="none" gap={1}> <list-item><icon name="check" color="success" inline/> Dipendenze installate correttamente</list-item> <list-item><icon name="check" color="success" inline/> Eseguite le otto configurazioni per tre semi</list-item> <list-item><icon name="check" color="success" inline/> Superati i controlli numerici programmati</list-item> <list-item><icon name="check" color="success" inline/> Caricato l'archivio dei risultati</list-item> </list> <text color="secondary" size="xs">Commit eseguito: `b9d371e3`. Commit attuale del ramo: `4e104cbc`. L'ultimo commit aggiunge documentazione ed evidenze, senza modificare il simulatore.</text> </box>

L'archivio dei risultati è attualmente disponibile su <Link url="https://github.com/massimilianonardi-ai/rumiai-dev-PoCs/actions/runs/37754200236/artifacts/11539771228" title="GitHub"/> e ha una scadenza prevista del **7 novembre 2026**.

Questa verifica conferma che non si tratta soltanto di codice scritto o di una simulazione dichiarata: esiste un'esecuzione reale documentata da GitHub.

---

3. Risultati scientifici

La metrica principale è NMSE: più è piccola, migliore è la previsione. Zero rappresenta un risultato praticamente perfetto.

<Chart content={{"chartType":"bar","meta":{"title":"Errore di previsione temporale (NMSE)","description":"Media di tre semi casuali. Valori minori indicano prestazioni migliori."},"xKey":"model","series":[{"dataKey":"nmse","label":"NMSE"}],"yAxisMin":0,"data":[{"model":"Quadratico digitale","nmse":0},{"model":"Ritardi digitali","nmse":0.561},{"model":"ESN digitale","nmse":0.812},{"model":"R-C lineare","nmse":0.832},{"model":"Diodi accoppiati","nmse":0.83},{"model":"Diodi indipendenti","nmse":0.845},{"model":"Ottico senza feedback","nmse":1.001},{"model":"Ottico con fibra","nmse":0.999}]}}/>

I risultati riportati dal workflow sono coerenti con il codice e con il rapporto conservato nel repository.

C'è una scoperta interessante: le celle a diodi riescono a rappresentare la funzione non lineare (u^2) con un NMSE di circa **0,148**, mentre la rete R-C lineare ottiene circa **1,002**.

Questo conferma che le non linearità del circuito generano informazioni utilizzabili dalla lettura digitale. È un risultato atteso dalla teoria, non ancora una capacità innovativa.

Sul compito temporale complessivo, invece, i modelli fisici simulati non superano i riferimenti digitali più semplici.

Un secondo risultato particolarmente importante riguarda la robustezza: applicando un rumore di lettura dell'1%, l'NMSE temporale della rete R-C passa da circa **0,83 a 110,88**. Significa che un modello apparentemente funzionante in condizioni ideali può diventare praticamente inutilizzabile quando introduciamo piccole imperfezioni.

4. Valutazione delle decisioni prese da Work

Decisione	Valutazione
Simulare prima di costruire hardware	<icon name="check-circle" color="success" inline/> Corretta
Usare ngspice per i circuiti elettronici	<icon name="check-circle" color="success" inline/> Corretta
Utilizzare un modello ottico semplificato	<icon name="check-circle" color="success" inline/> Appropriata per una prima esplorazione
Confrontare modelli fisici e digitali	<icon name="check-circle" color="success" inline/> Necessaria
Mantenere il PoC fuori da `rumiai-os`	<icon name="check-circle" color="success" inline/> Conforme all'architettura
Utilizzare Python	<icon name="check-circle" color="success" inline/> Giustificato dalle librerie scientifiche utilizzate
Verificare su GitHub Actions	<icon name="check-circle" color="success" inline/> Esecuzione confermata
Sincronizzare lo stato nel handoff	<icon name="alert-triangle" color="warning" inline/> Non completato

La scelta di Python merita una precisazione: le regole correnti RumiAI lo sconsigliano per il nuovo software proprietario del progetto, ma non lo vietano. In questo caso Work ha documentato una motivazione specifica legata a SciPy e NumPy, quindi non la considero una violazione del contratto.

5. Problemi e limiti identificati

<box gap={3}> <row align="start" gap={2}> <icon name="alert-triangle" color="danger" size="lg"/> <box flex="1" gap={1}> **Priorità alta — Handoff non sincronizzato**

```
  Il file <Link url="https://github.com/massimilianonardi-ai/rumiai-
dev/blob/main/handoff/chaotic-dynamics-rumiai-research.md" title="chaotic-dynamics-
rumiai-research.md"/> è fermo al 7 ottobre e continua a dichiarare che nessuna
simulazione è stata eseguita.

  Questo contraddice lo stato del PoC pubblicato l'8 ottobre. Le regole RumiAI
richiedono di aggiornare l'handoff dopo risultati sperimentali significativi.
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={2}> <icon name="alert-circle" color="warning" size="lg"/> <box flex="1" gap={1}> **Priorità media — Confronti ancora limitati**

```
  Il regressore quadratico digitale dispone di 152 caratteristiche, mentre i
principali modelli fisici ne espongono 16. Work lo dichiara correttamente, ma il
confronto non dimostra quale soluzione sarebbe più efficiente a parità di hardware,
energia o costo.

  Anche le architetture non sono state ottimizzate con un budget di ricerca
comparabile.
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={2}> <icon name="alert-circle" color="warning" size="lg"/> <box flex="1" gap={1}> **Priorità media — Copertura incompleta delle idee iniziali**

```
  Il PoC non comprende ancora neuroni impulsivi, plasticità locale, apprendimento
distribuito, analisi quantitativa del caos, sincronizzazione o reti ottiche complesse.

  Non lo considero un errore di implementazione: è una riduzione ragionevole del campo
sperimentale. Tuttavia, questi temi rappresentano ancora una parte importante
dell'obiettivo originale.
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={2}> <icon name="alert-circle" color="warning" size="lg"/> <box flex="1" gap={1}> **Priorità media — Nessuna validazione fisica**

```
  L'accordo tra ngspice e SciPy dimostra coerenza fra due modelli numerici. Non
dimostra ancora che un circuito reale rispetterà quelle equazioni con la precisione
desiderata.

  Il modello ottico, in particolare, trascura diverse proprietà reali di laser,
modulatori e fibre.
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={2}> <icon name="info" color="secondary" size="lg"/> <box flex="1" gap={1}> **Priorità bassa — Conservazione delle evidenze**

```
  L'archivio GitHub occupa circa 738 MB e scadrà dopo 30 giorni. I rapporti sintetici
restano nel repository, ma conviene rendere più selettiva la conservazione delle
traiettorie complete.
</box>
```

</row> </box>

---

6. Che cosa possiamo concludere sulla sessione Work

La trascrizione condivisa contiene le richieste, le spiegazioni e gli aggiornamenti operativi di Work, ma non il contenuto integrale dei suoi comandi e delle sue operazioni interne. Posso quindi ricostruire le scelte dichiarate e verificarne gli effetti su GitHub, non certificare ogni passaggio dell'interazione.

Ho verificato il codice, la configurazione del workflow, il job concluso, i log, i risultati e il confronto tra il commit eseguito e quello finale. **Non ho effettuato una nuova esecuzione indipendente del simulatore** né una validazione fisica.

La revisione dello stato corrente è stata effettuata in sola lettura: nessun repository è stato modificato.

---

7. Come procederei

La mia proposta è conservare il PoC 058 come prima base sperimentale, senza interpretarne i risultati negativi come un fallimento dell'intera ricerca.

Prima di espanderlo, completerei tre attività:

1. **Riallineare l'handoff** con il PoC 058, i risultati effettivi e le questioni ancora aperte.
2. **Rendere più rigoroso il confronto**, introducendo risorse comparabili, controlli di robustezza e un problema temporale meno costruito intorno ai riferimenti digitali.
3. **Preparare un secondo esperimento** dedicato proprio all'ambizione principale: reti analogiche con stati interni distribuiti, apprendimento locale, più scale temporali e dinamiche collettive, per capire se emergono proprietà utili che i modelli convenzionali non ottengono alle stesse condizioni.

Giudizio complessivo: Work ha svolto un buon primo lavoro sperimentale, concretamente verificabile. La maggiore qualità della sessione è avere riportato risultati negativi anziché inventare un successo. La principale lacuna operativa è l'handoff non aggiornato; la principale lacuna scientifica è che abbiamo esplorato alcune forme di reservoir computing, ma non ancora le architetture neuromorfiche più ambiziose che ti interessano.

Il PoC è una base valida per continuare la ricerca, non ancora una dimostrazione di un nuovo acceleratore AI.

USER

che significa che l'archivio github scade dopo 30 giorni? quale archivio?

ASSISTANT

Si tratta dell'**artifact generato da GitHub Actions durante l'esecuzione del PoC 058**, non del repository GitHub.

Quando Work ha eseguito il simulatore su una VM GitHub, il programma ha prodotto numerosi file di risultati. GitHub li ha raccolti in un archivio ZIP scaricabile.

Cosa contiene questo archivio?

- I circuiti elettronici generati per ngspice (`circuit.cir`).
- I log delle simulazioni.
- Le traiettorie delle tensioni simulate (`states.dat`).
- I segnali utilizzati nei test.
- Le previsioni ottenute dai modelli.
- I risultati numerici completi (`results.json`).
- Il rapporto riepilogativo (`report.md`).

<box border radius="lg" padding={3} gap={2}> <row align="center" gap={2}> <icon name="file-archive" color="secondary" size="lg"/> <box flex="1" gap="1px"> **Archivio PoC 058** <text color="secondary" size="xs">GitHub Actions · Run 37754200236</text> </box> </row> <divider color="subtle"/> <row justify="between"> <text color="secondary" size="sm">Dimensione</text> **738 MB circa** </row> <row justify="between"> <text color="secondary" size="sm">Creato</text> **8 ottobre 2026** </row> <row justify="between"> <text color="secondary" size="sm">Scadenza prevista</text> **7 novembre 2026** </row> <row justify="between" align="center"> <text color="secondary" size="sm">Stato verificato</text> <badge color="success">Disponibile</badge> </row> <button block color="primary" onClick={()=>GenUI.openUrl("https://github.com/massimilianonardi-ai/rumiai-dev-PoCs/actions/runs/37754200236/artifacts/11539771228")}> <icon name="external-link" inline/> Apri archivio su GitHub </button> </box>

Cosa succede dopo 30 giorni?

GitHub elimina automaticamente l'artifact alla scadenza della conservazione configurata per il workflow.

Non vengono eliminati:

- Il codice del PoC 058.
- Il ramo Git e i relativi commit.
- I README e i rapporti sintetici salvati nel repository.
- I risultati numerici della simulazione SciPy che Work ha già conservato in Git.

Vengono invece persi dall'archivio GitHub Actions i file dettagliati di quell'esecuzione, come le traiettorie elettroniche complete, se non sono stati conservati altrove.

Il simulatore potrà comunque essere rieseguito per produrre nuovi risultati.

Cosa suggerisco per RumiAI

Distinguerei due tipologie di dati:

1. **Risultati scientifici importanti:** dovrebbero essere conservati permanentemente, con parametri, revisioni, misure e informazioni sufficienti a riprodurre gli esperimenti.
2. **Dati intermedi voluminosi:** possono essere temporanei, purché rigenerabili e non indispensabili per verificare le conclusioni.

Per il PoC 058, ad esempio, non è necessario salvare permanentemente 738 MB di dati per ogni esecuzione, ma è importante preservare le evidenze necessarie a verificarne i risultati.

Quindi non rischiamo di perdere il PoC. Rischiamo soltanto di perdere i dati dettagliati di quella specifica esecuzione, che GitHub conserva temporaneamente.

USER

cosa proponi?

ASSISTANT

Propongo di adottare per i PoC RumiAI una strategia molto semplice: **conservare permanentemente i risultati scientifici e tutto ciò che serve a riprodurli, senza accumulare gigabyte di dati intermedi.**

Nel PoC 058 abbiamo già quasi tutto il necessario. Modificherei soltanto la gestione delle evidenze.

La soluzione che sceglierei

Dati	Conservazione	Dove
Codice, modelli e parametri	Permanente	Git
Risultati numerici (`results.json`)	Permanente	Git
Rapporto scientifico (`report.md`)	Permanente	Git
Versioni software, commit e controlli	Permanente	Git
Traiettorie complete (`states.dat`)	Temporanea	GitHub Actions
Log dettagliati e file intermedi	Temporanea	GitHub Actions

Interventi concreti

1. **Salvare anche il `results.json` dell'esecuzione ngspice.** Attualmente nel repository è conservato il rapporto sintetico, mentre il JSON completo dell'esecuzione GitHub è soltanto nell'artifact temporaneo.
2. **Ridurre l'artifact GitHub.** Anziché includere tutte le traiettorie, salverei per impostazione predefinita risultati, log diagnostici essenziali e un piccolo campione di segnali utile per verificare il comportamento.
3. **Permettere una modalità diagnostica completa.** Quando un esperimento presenta anomalie, il workflow potrà conservare anche tutte le traiettorie per 30 giorni. Non servono per ogni esecuzione.
4. **Garantire la riproducibilità.** Ogni esperimento conserverà revisione esatta, configurazione, semi casuali, versioni e risultati. Non basta poter rieseguire il programma: dobbiamo poter confrontare la nuova esecuzione con quella originale.
5. **Aggiornare l'handoff RumiAI.** Deve finalmente indicare che il PoC 058 è stato eseguito, con i suoi risultati e limiti.

Perché preferisco questa soluzione

Non creerei un database, un archivio esterno o un sistema complesso di gestione dei risultati.

Il PoC 058 è ancora piccolo e sperimentale. Il suo valore è nella conoscenza prodotta, non nei 738 MB di file intermedi.

Un dettaglio importante: **le nuove esecuzioni non sostituiscono l'evidenza originale**. Perciò manterrei permanentemente i risultati numerici effettivamente ottenuti da ngspice, anziché affidarmi soltanto alla possibilità di rigenerarli.

In questo modo possiamo anche eseguire centinaia di esperimenti senza far crescere inutilmente i repository.

La mia proposta è applicare questi cinque interventi direttamente al PoC 058, senza modificare l'architettura del simulatore e senza introdurre nuove infrastrutture. La verifica degli HEAD e delle regole correnti è stata completata; non ho ancora modificato nulla.

USER

ok procedi

ASSISTANT

Ho completato il recupero dell'evidenza originale del PoC 058.

- **Il `results.json` ngspice originale è ora conservato permanentemente in Git**, sul ramo sperimentale. Ho verificato che provenga dall'archivio dell'8 ottobre mediante SHA-256, senza rigenerarlo.
- Ho modificato il workflow affinché carichi normalmente soltanto risultati, metadati, previsioni e brevi campioni diagnostici.
- Ho aggiunto l'opzione `full_diagnostics` per richiedere esplicitamente l'archivio completo.
- Ho eliminato dal ramo il workflow temporaneo utilizzato per il recupero.

Le modifiche sono nel commit `0a9663f`. `main` è rimasto invariato.

La nuova esecuzione su GitHub Actions è terminata con successo.

<grid columns={2} gap={3}> <grid-item> <text color="secondary" weight="medium" size="xs">Archivio originale</text> <title size="xl" color="default" tabularNums>738 MB</title> </grid-item> <grid-item> <text color="secondary" weight="medium" size="xs">Nuovo archivio compatto</text> <title size="xl" color="success" tabularNums>3,19 MB</title> </grid-item> </grid>

La dimensione dell'artifact è diminuita di circa **99,6%**. Il programma completo è stato eseguito ancora, ma GitHub conserva temporaneamente soltanto la selezione essenziale.

Puoi consultare la nuova esecuzione GitHub Actions.

Ho completato anche il riallineamento della documentazione RumiAI.

Il file `handoff/chaotic-dynamics-rumiai-research.md` ora registra le simulazioni eseguite, i risultati scientifici, le revisioni coinvolte e le attività di ricerca ancora aperte.

Riferimenti finali:

- PoC 058 — Draft PR #1
- Commit di aggiornamento dell'handoff
- Risultati ngspice originali conservati in Git

Consistency check finale completato: revisioni remote verificate, modifiche controllate e nuovo workflow eseguito con successo. Nessuna modifica al modello matematico e nessuna validazione fisica dichiarata.

Rimane da provare separatamente soltanto la modalità opzionale `full_diagnostics`; la modalità compatta è già validata.

Il lavoro resta nel ramo sperimentale, con PR aperta in modalità draft. Nessun merge è stato effettuato.

USER

come è modellata chatgpt?

ASSISTANT

ChatGPT è basata su una **rete neurale artificiale di grandi dimensioni**, appartenente alla famiglia dei *Transformer*. È un modello matematico che apprende relazioni tra informazioni modificando un numero molto elevato di parametri numerici.

La cosa interessante, pensando alla nostra ricerca sul calcolo analogico, è che **gran parte delle operazioni interne di un Transformer sono matematicamente continue, anche se vengono eseguite su circuiti digitali**.

---

1. Come è organizzata

<box border radius="xl" padding={3} gap={2}> <box background="surface-secondary" padding={3} radius="lg" align="center"> **Input: testo, immagini e altre informazioni** </box> <row justify="center"><icon name="arrow-down" color="secondary" size="lg"/></row> <box border radius="lg" padding={3} align="center" gap={1}> **Codifica in rappresentazioni numeriche**

```
<text color="secondary" size="xs">Token, embedding, informazioni di posizione</text>
```

</box> <row justify="center"><icon name="arrow-down" color="secondary" size="lg"/></row> <box background="rgba(55,118,205,0.06)" border radius="lg" padding={3} gap={2}> <text color="default" weight="medium" textAlign="center">Blocchi neurali ripetuti molte volte</text> <grid columns={2} gap={2}> <grid-item> <box background="surface" border radius="md" padding={2} align="center" justify="center" minHeight="80px"> **Attention** <text color="secondary" textAlign="center" size="xs">Relazioni tra elementi del contesto</text> </box> </grid-item> <grid-item> <box background="surface" border radius="md" padding={2} align="center" justify="center" minHeight="80px"> **Rete non lineare** <text color="secondary" textAlign="center" size="xs">Trasformazione delle rappresentazioni</text> </box> </grid-item> </grid> <text color="secondary" textAlign="center" size="xs">Connessioni residue e normalizzazioni accompagnano le trasformazioni</text> </box> <row justify="center"><icon name="arrow-down" color="secondary" size="lg"/></row> <box border radius="lg" padding={3} align="center" gap={1}> **Probabilità dei possibili token successivi** </box> <row justify="center"><icon name="arrow-down" color="secondary" size="lg"/></row> <box background="surface-secondary" radius="lg" padding={3} align="center"> **Output generato progressivamente** </box> </box>

Questo schema rappresenta un Transformer generativo in modo semplificato, non la descrizione completa dell'architettura proprietaria di GPT-6. OpenAI ha documentato pubblicamente l'impiego di modelli di tipo Transformer e la generazione autoregressiva, ma non tutti i dettagli implementativi dei modelli più recenti. <Cite refs={["turn458525search24","turn458525search0"]}/>

---

2. Come funziona matematicamente

Il calcolo interno consiste soprattutto in quattro operazioni:

Operazione	Formula esemplificativa	Possibile equivalente fisico
Somma pesata	(y=\sum_i w_i x_i)	Somma di correnti
Trasformazione non lineare	(y=f(x))	Transistor, diodi
Memoria delle informazioni contestuali	Stati e vettori K/V	Memorie elettroniche, altre tecnologie di accumulo
Attention	(\operatorname{softmax}(QK^T/\sqrt{d})V)	Matrici di calcolo, eventualmente analogiche o ottiche

L'**attention** è particolarmente importante. Permette al modello di calcolare dinamicamente quali elementi del contesto sono rilevanti per elaborare le informazioni successive. <Cite ref="turn458525search3"/>

---

3. In cosa differisce dal cervello biologico?

Cervello	Modello Transformer
Neuroni biologici con dinamiche temporali	Unità matematiche organizzate in livelli
Segnali elettrochimici continui e impulsi	Vettori numerici e operazioni tensoriali
Connessioni fisiche modificabili	Pesi appresi durante l'addestramento
Memoria distribuita in molte forme	Pesi, contesto e stati temporanei di calcolo
Elaborazione continua e fortemente ricorrente	Elaborazione prevalentemente a livelli, con generazione autoregressiva

Un punto fondamentale: **ChatGPT non simula fisicamente il funzionamento dei neuroni biologici.** Utilizza una particolare struttura matematica che si è dimostrata efficace nell'apprendere e generare informazioni.

Inoltre, durante una normale conversazione, i pesi del modello non vengono aggiornati continuamente come le sinapsi del cervello. Le informazioni del dialogo influenzano il calcolo attraverso il contesto.

---

4. Cosa significa per la nostra ricerca hardware?

Vedo due direzioni molto diverse.

A. Accelerare un Transformer esistente. Potremmo studiare circuiti analogici, elettronici o fotonici per moltiplicazioni matriciali, somme e trasformazioni non lineari. L'obiettivo sarebbe eseguire lo stesso modello con un rapporto migliore tra energia, velocità e costo. Resterebbero da risolvere precisione, conversione dei segnali e memoria.

B. Studiare un modello di intelligenza differente. Potremmo creare reti fisiche in cui ogni elemento mantiene uno stato, comunica continuamente con altri elementi e modifica localmente le proprie connessioni. Sarebbe molto più vicino alla direzione neuromorfica che stiamo esplorando, ma non è dimostrato che un simile sistema possa raggiungere le capacità di un grande Transformer.

Questa seconda direzione non deve necessariamente usare token, attention o livelli sequenziali. Potrebbe sfruttare direttamente la dinamica collettiva del sistema.

La distinzione più importante è questa: il Transformer descrive *quali operazioni matematiche eseguire*; il nostro progetto può indagare sia *come eseguirle con una fisica differente*, sia *se esista un modello computazionale migliore da realizzare fisicamente*.

Per approfondire, la domanda che trovo più interessante è: **quanta parte dell'intelligenza di ChatGPT dipende davvero dall'architettura Transformer e quanta dall'apprendimento, dalla memoria e dalla quantità di risorse utilizzate?** È proprio da questa distinzione che partirei per confrontare un'eventuale architettura analogica con i modelli attuali.

USER

io sono orientato al percorrere la strada B, ma se analizziamo le ai open source migliori e troviamo una strada per A, allora proviamo anche quella.

ASSISTANT

Concordo con questa impostazione. Per RumiAI terrei **B come direzione principale della ricerca hardware**: non limitarci ad accelerare i modelli esistenti, ma cercare una forma di calcolo nella quale memoria, elaborazione e apprendimento emergano dalle dinamiche della rete fisica.

La strada **A rimane aperta**, ma con una condizione: adottarla soltanto quando individuiamo in un modello AI aperto un'operazione specifica che possiamo realizzare in modo fisicamente vantaggioso.

Le due ricerche possono anche aiutarsi reciprocamente. Studiare le architetture dei modelli attuali ci permette di capire quali meccanismi sono davvero importanti e quali potrebbero essere riprodotti attraverso reti analogiche.

Una prima selezione di architetture da studiare

Modello	**Caratteristica interessante**	**Rilevanza**
<Entity category="software" value="RWKV-7" disambig="recurrent neural language model architecture"/>	Stato ricorrente di dimensione costante, senza attention tradizionale	**B:** memoria e dinamiche locali
<Entity category="software" value="Qwen3.5" disambig="Alibaba open-weight language model family"/>	Alterna Gated DeltaNet e attention, con varianti MoE	**A e B:** stati ricorrenti e calcolo matriciale
<Entity category="software" value="DeepSeek-V4" disambig="DeepSeek open-weight mixture-of-experts language model"/>	Attention ibrida, routing degli esperti e connessioni avanzate	**A:** individuare operazioni accelerabili
<Entity category="software" value="llama.cpp" disambig="C and C++ local AI inference engine"/>	Motore aperto con backend hardware sostituibili	**A:** sperimentare un acceleratore senza riprogettare l'intera AI

Un esempio particolarmente pertinente è **RWKV-7**: il modello ufficiale dichiara uno stato ricorrente di dimensione costante durante la generazione. Qwen3.5 usa invece una combinazione di tre blocchi Gated DeltaNet e uno di attention completa. Sono due modi concreti, già implementati, di affrontare memoria e sequenze senza affidarsi

esclusivamente alla classica attention Transformer. <Cite refs={["turn855189search8","turn855189search1","turn855189search4"]}/>

Non significa che questi modelli siano già equivalenti a reti neuromorfiche. Significa che sono ottime architetture da cui estrarre **meccanismi matematici verificabili**.

Come organizzerei i prossimi esperimenti

<box gap={3}> <row align="start" gap={3}> <box background="surface-secondary" radius="lg" padding={2}> <icon name="brain" size="lg"/> </box> <box flex="1" gap={1}> **Percorso B — Architetture cognitive fisiche**

```
  Continuerei dagli esperimenti già avviati con il PoC 059: neuroni caotici, rumore,
accoppiamento e sincronizzazione. Il passo successivo sarà verificare se queste
proprietà migliorano davvero un compito di apprendimento o memoria, confrontandole con
reti non caotiche e modelli digitali.
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={3}> <box background="surface-secondary" radius="lg" padding={2}> <icon name="cpu" size="lg"/> </box> <box flex="1" gap={1}> **Percorso A — Accelerazione selettiva**

```
  Come primo candidato sceglierei una proiezione matriciale di un modello aperto
relativamente piccolo, ad esempio Qwen3.5-4B. Simuleremmo una realizzazione analogica
con precisione limitata, rumore e deriva, confrontandola con il calcolo digitale
quantizzato.

  La sfida sarà dimostrare un vantaggio considerando anche conversioni ADC/DAC,
accesso ai pesi e calibrazione: sono costi che la ricerca sui processori fotonici
identifica come cruciali. <Cite refs={["turn221144search0","turn497832search8"]}/>
</box>
```

</row> </box>

Dalla prima analisi delle fonti ufficiali emerge che **RWKV-7 e Qwen3.5 meritano particolare attenzione**, non perché siano necessariamente i modelli più potenti in assoluto, ma perché offrono meccanismi verificabili di memoria ricorrente e codici/pesi accessibili. Le versioni esaminate dichiarano licenze Apache-2.0; per DeepSeek-V4 sono disponibili anche implementazioni documentate di attention ibrida e proiezioni a basso rango. <Cite refs={["turn651965view0","turn651965view1","turn221144view1"]}/>

Ho registrato questa priorità e i criteri di valutazione nel handoff corrente di ricerca RumiAI, con il commit `6f22b79`.

La verifica finale del diff e dei riferimenti è completata. Nessuna architettura di prodotto o simulatore è stata modificata in questo passaggio, quindi non sono stati necessari nuovi test eseguibili.

La direzione rimane B. La ricerca A diventa un modo per individuare opportunità concrete, senza distrarre risorse dalla sperimentazione di modelli di calcolo alternativi.

USER

esiste qualche implementazione moderna di ai che in qualche modo riprenda il vecchio modello del perceptron multistrato?

ASSISTANT

Sì. E alcune implementazioni moderne riprendono il perceptron multistrato (MLP) in modi molto interessanti, anche al di fuori dei Transformer.

Il vecchio MLP non è stato abbandonato: è stato ampliato con nuove funzioni di attivazione, connessioni residue, meccanismi di memoria e, in alcuni casi, apprendimento durante l'esecuzione.

Per la nostra ricerca sul calcolo analogico, distinguerei due famiglie: **reti che modernizzano il MLP tradizionale** e **reti che ne conservano i neuroni e le connessioni, ma introducono dinamiche temporali o apprendimento locale**.

Le implementazioni che studierei

<box gap={3}> <row align="start" gap={3}> <AsyncImage query="MLP Mixer neural network token mixing channel mixing architecture diagram" aspectRatio="4:3" maxWidth="128px"/> <box flex="1" gap={1}> **1. <Entity category="software" value="MLP-Mixer" disambig="Google Research all-MLP visual recognition architecture"/> (2021)**

```
  È probabilmente l'erede più diretto del MLP classico. Utilizza esclusivamente reti
MLP per elaborare le immagini, senza convoluzioni né self-attention. Ha dimostrato che
reti di questo tipo possono essere competitive nella classificazione visiva. <Cite
refs={["turn495806search0","turn495806search1"]}/>

  <text color="secondary" size="sm">Codice: <Link url="https://github.com/google-
research/vision_transformer" title="Google Research"/></text>
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={3}> <AsyncImage query="gMLP spatial gating unit neural network architecture diagram" aspectRatio="4:3" maxWidth="128px"/> <box flex="1" gap={1}> **2. gMLP (2021)**

```
  Introduce un meccanismo di *gating*, cioè elementi della rete che modulano
l'attività di altri elementi. È stato sperimentato sia nella visione sia
nell'elaborazione del linguaggio, raggiungendo risultati competitivi con i Transformer
sui compiti studiati. <Cite ref="turn495806academia62"/>

  <text color="secondary" size="sm">Ricerca: <Link
url="https://arxiv.org/abs/2105.08050" title="Pay Attention to MLPs"/></text>
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={3}> <AsyncImage query="SIREN sinusoidal activation implicit neural representation sine network visualization" aspectRatio="4:3" maxWidth="128px"/> <box flex="1"

gap={1}> **3. <Entity category="software" value="SIREN" disambig="Sinusoidal representation neural network architecture"/> (2020)**

```
  È una rete MLP che utilizza funzioni di attivazione sinusoidali. Può rappresentare
immagini, audio, campi continui e soluzioni di equazioni differenziali.

  È interessante per noi perché l'oscillazione è una proprietà naturale di molti
circuiti elettronici e ottici. <Cite ref="turn155167search0"/>

  <text color="secondary" size="sm">Codice: <Link
url="https://github.com/vsitzmann/siren" title="Implementazione ufficiale"/></text>
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={3}> <AsyncImage query="Deep equilibrium model implicit neural network fixed point recurrent architecture diagram" aspectRatio="4:3" maxWidth="128px"/> <box flex="1" gap={1}> **4. <Entity category="software" value="Deep Equilibrium Models" disambig="implicit neural network fixed-point architecture"/> (DEQ, 2019)**

```
  Anziché attraversare molti livelli distinti, la rete applica ripetutamente una
trasformazione finché raggiunge uno stato di equilibrio.

  È vicina al comportamento di una rete fisica con retroazione che evolve
spontaneamente verso uno stato stabile. <Cite ref="turn495806search2"/>

  <text color="secondary" size="sm">Codice: <Link
url="https://github.com/locuslab/deq" title="locuslab/deq"/></text>
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={3}> <AsyncImage query="test time training TTT MLP recurrent neural network hidden state learning diagram" aspectRatio="4:3" maxWidth="128px"/> <box flex="1" gap={1}> **5. TTT-MLP (2024)**

```
  Questo è particolarmente interessante: lo stato interno della rete ricorrente è esso
stesso un piccolo MLP a due livelli, i cui pesi vengono aggiornati mentre il modello
elabora nuovi dati.

  Invece di ricordare soltanto mediante un vettore di attivazioni, il sistema conserva
informazione anche attraverso l'adattamento dei parametri della propria memoria. <Cite
refs={["turn155167search3","turn155167search5"]}/>

  <text color="secondary" size="sm">Codice: <Link url="https://github.com/test-time-
training/ttt-lm-pytorch" title="TTT-LM PyTorch"/></text>
</box>
```

</row> </box>

Quali sono più interessanti per RumiAI?

Architettura	Legame con MLP	Potenziale per calcolo fisico
MLP-Mixer	Diretto	Somme pesate e parallelismo
gMLP	Diretto, con gating	Modulazione e inibizione
SIREN	Diretto, con sinusoidi	Oscillatori elettronici/ottici
DEQ	Trasformazione ripetuta fino all'equilibrio	Reti analogiche ricorrenti
TTT-MLP	MLP come memoria adattiva	Connessioni modificabili nel tempo

La proprietà che trovo più importante non è la semplicità del MLP, ma il fatto che possiamo passare da una rete statica:

[y=f(Wx+b)]

a una rete con stato e retroazione:

[\tau\frac{dx}{dt}=-x+f(Wx+Bu)]

e, successivamente, a collegamenti che cambiano nel tempo:

[\frac{dW}{dt}=F(x,u,W)]

Quest'ultima formula è una famiglia generica di modelli, non un algoritmo di apprendimento già definito. La funzione (F) sarebbe proprio uno dei problemi principali da studiare.

In una rete fisica, i valori (x) potrebbero essere tensioni, correnti, stati ottici o altre grandezze; i pesi (W) potrebbero essere conduttanze regolabili o altri elementi che controllano l'accoppiamento.

Un'altra ricerca particolarmente pertinente: Equilibrium Propagation

L'<Entity category="scientific_concept" value="Equilibrium Propagation" disambig="learning method for energy-based neural networks"/> è un metodo di apprendimento per reti dinamiche basate su stati di equilibrio. Permette di studiare aggiornamenti dei collegamenti attraverso variazioni locali tra stati della rete, invece di eseguire necessariamente la normale retropropagazione digitale attraverso molti livelli.

Un lavoro pubblicato su *Nature Communications* nel 2024 ha dimostrato l'addestramento supervisionato di una rete neurale implementata su una macchina Ising mediante Equilibrium Propagation, con esperimenti sul dataset MNIST. Non è la dimostrazione di un'intelligenza generale analogica, ma è una prova concreta del collegamento tra dinamiche fisiche e apprendimento. <Cite ref="turn155167search10"/>

Per questo, **approfondirei per primi DEQ, TTT-MLP ed Equilibrium Propagation**, tenendo SIREN come riferimento per neuroni oscillatori. MLP-Mixer rimarrebbe un buon riferimento per confrontare il calcolo analogico con reti feed-forward classiche.

La domanda sperimentale più promettente sarebbe: *possiamo costruire una rete di neuroni fisici che mantenga uno stato, raggiunga un equilibrio o un regime dinamico utile e apprenda modificando localmente le proprie connessioni?*

Questo sarebbe un passo sostanziale verso la strada B che hai scelto, senza dover riprodurre un Transformer.

USER

confronta mlp con ai open source che hai menzionato prima

ASSISTANT

La differenza fondamentale è questa: **il perceptron multistrato (MLP) e le AI aperte come RWKV-7, Qwen3.5 e DeepSeek-V4 non sono tecnologie completamente alternative.** I modelli moderni utilizzano ancora reti di tipo MLP al loro interno, ma aggiungono meccanismi per gestire memoria, sequenze, contesto e specializzazione.

Per RumiAI è una distinzione importante: potremmo partire dalla semplicità del MLP e studiare quali proprietà delle AI moderne aggiungere, senza necessariamente riprodurre un Transformer completo.

---

1. Confronto delle architetture

Architettura	Elaborazione	Memoria durante l'inferenza	Gestione del linguaggio
MLP classico	Livelli di neuroni e connessioni pesate	Nessuno stato persistente intrinseco	Possibile, ma limitata senza altri meccanismi
MLP-Mixer / gMLP	MLP con mescolamento delle informazioni o gating	Generalmente nessuna memoria ricorrente	gMLP è stato sperimentato anche sul linguaggio
TTT-MLP	Piccolo MLP adattato durante l'elaborazione	Pesi del modello interno aggiornabili	Progettato per sequenze linguistiche
RWKV-7	Rete ricorrente con aggiornamento dinamico dello stato	Stato ricorrente che passa da un token al successivo	Modello linguistico generativo
Qwen3.5	Architettura ibrida: Gated DeltaNet, attention e reti feed-forward	Stati ricorrenti e cache dell'attention	Modello linguistico e multimodale
DeepSeek-V4	Transformer ibrido con esperti specializzati	Cache e rappresentazioni del contesto	Modello linguistico generativo di grande scala

MLP-Mixer è un esempio dimostrato di architettura quasi interamente costruita con MLP per la visione. TTT-MLP, invece, utilizza un MLP proprio come memoria che viene aggiornata durante l'elaborazione. RWKV-7 introduce dinamiche ricorrenti; Qwen3.5 combina ricorrenza e attention; DeepSeek-V4 aggiunge specializzazione tramite *Mixture of Experts*. <Cite refs=
{["turn868632search3","turn439379search13","turn439379search0","turn439379search7","turn868632search1"]}/>

Non sarebbe corretto dedurre da questa tabella una classifica di intelligenza. Un MLP piccolo addestrato per classificare immagini e un modello linguistico addestrato su enormi quantità di dati non sono direttamente confrontabili. Le prestazioni dipendono anche da dimensione, dati, addestramento e risorse utilizzate.

2. Che cosa aggiungono davvero le AI moderne al MLP?

Un MLP tradizionale calcola:

[h=f(Wx+b)]

I pesi (W) contengono ciò che è stato appreso durante l'addestramento. Quando riceve un nuovo input, però, la rete normalmente riparte senza una memoria dinamica delle elaborazioni precedenti.

Le AI che abbiamo considerato introducono tre meccanismi particolarmente importanti.

<box gap={3}> <row align="start" gap={3}> <box background="surface-secondary" padding={2} radius="lg"> <icon name="refresh-cw" size="lg"/> </box> <box flex="1" gap={1}> **RWKV-7 — Memoria ricorrente**

```
  Aggiunge uno stato che evolve con la sequenza. La sua caratteristica interessante è
poter utilizzare una memoria ricorrente di dimensione non crescente con la lunghezza
del testo, diversamente da una cache di attention tradizionale. <Cite
ref="turn439379search0"/>

  Non è più un semplice MLP, ma molte operazioni interne sono ancora trasformazioni
lineari e non lineari.
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={3}> <box background="surface-secondary" padding={2} radius="lg"> <icon name="network" size="lg"/> </box> <box flex="1" gap={1}> **Qwen3.5 — Memoria e attenzione combinate**

```
  Per esempio, Qwen3.5-4B utilizza 32 livelli, alternando blocchi Gated DeltaNet e
attention completa. Ciascun blocco contiene anche una rete feed-forward,
matematicamente molto vicina a un MLP. <Cite ref="turn707570view0"/>

  È un riferimento utile per capire come integrare trasformazioni MLP con stati
ricorrenti e accesso selettivo al contesto.
</box>
```

</row> <divider color="subtle"/> <row align="start" gap={3}> <box background="surface-secondary" padding={2} radius="lg"> <icon name="git-branch" size="lg"/> </box> <box flex="1" gap={1}> **DeepSeek-V4 — Molte reti specializzate**

```
  Utilizza una struttura Mixture-of-Experts: diversi sottosistemi feed-forward
specializzati, dei quali soltanto una parte viene attivata durante una determinata
elaborazione. <Cite refs={["turn868632search1","turn868632search4"]}/>

  Il principio interessante è che non tutti i neuroni o i moduli devono intervenire
sempre. In prospettiva potrebbe suggerire reti fisiche con attivazione selettiva,
anche se il routing digitale degli esperti non equivale a un'attivazione neuromorfica
autonoma.
</box>
```

</row> </box>

C'è quindi una differenza tra **la capacità di elaborare informazione di un neurone** e **la capacità della rete di conservare, selezionare e modificare informazione nel tempo**.

I grandi modelli moderni investono moltissima complessità soprattutto nel secondo aspetto.

---

3. Qual è più interessante per costruire un'AI fisica?

Questa è la mia valutazione progettuale, non una graduatoria sperimentale già dimostrata.

Architettura	**Facilità di realizzazione analogica iniziale**	**Interesse per la strada B**
MLP classico	Alta	Medio
SIREN (MLP sinusoidale)	Medio-alta	Medio-alto
MLP con retroazione	Alta per piccoli circuiti	Molto alto
Deep Equilibrium Models	Media	Molto alto
TTT-MLP	Bassa, per gli aggiornamenti dei pesi	Molto alto
RWKV-7	Media per alcune operazioni, bassa per il sistema completo	Alto
Qwen3.5	Bassa per il sistema completo	Medio, come riferimento
DeepSeek-V4	Molto bassa per il sistema completo	Medio, come riferimento

L'aspetto più promettente di **TTT-MLP** è che la memoria interna è un modello che viene aggiornato mentre arrivano nuovi token. Non è soltanto una rappresentazione temporanea mantenuta nei neuroni. Il lavoro originale identifica però proprio i costi di accesso alla memoria come una difficoltà importante. <Cite ref="turn439379academia97"/>

I **Deep Equilibrium Models** sono invece interessanti per un motivo differente: la rete raggiunge uno stato determinato da un'equazione di punto fisso, anziché attraversare soltanto un numero prestabilito di livelli. Questo comportamento si presta, almeno concettualmente, a essere studiato con circuiti fisici dotati di retroazione. <Cite ref="turn439379search4"/>

---

4. La possibile sintesi per RumiAI

Per la strada B non partirei cercando di replicare integralmente RWKV-7 o Qwen3.5 in analogico.

Partirei invece da un MLP tradizionale, mantenendo la semplicità delle sue connessioni, e aggiungerei gradualmente alcune proprietà:

<box border radius="lg" padding={3} gap={2}> <svg viewBox="0 0 340 190" width="100%" xmlns="http://www.w3.org/2000/svg"> {#each [0,1,2] as i} {#each [0,1,2,3] as j} <line x1={45} y1={49+i*43} x2={163} y2={31+j*37} stroke="currentColor" strokeWidth="0.8" opacity="0.12"/> {/each} {/each} {#each [0,1,2,3] as i} {#each [0,1,2] as j} <line x1={163} y1={31+i*37} x2={292} y2={49+j*43} stroke="currentColor" strokeWidth="0.8" opacity="0.12"/> {/each} {/each} {#each [0,1,2] as i} <circle cx="45" cy={49+i*43} r="11" fill="#64748b"/> <circle cx="292" cy={49+i*43} r="11" fill="#0d9488"/> {/each} {#each [0,1,2,3] as i} <circle cx="163" cy={31+i*37} r="12"

fill="#2563eb"/> {/each} <path d="M 163 19 C 205 -2, 228 8, 224 48 C 222 67, 204 68, 181 64" stroke="#d97706" strokeWidth="2.5" strokeDasharray="5 3" fill="none"/> <path d="M181 64 L190 53 L192 68 Z" fill="#d97706"/> <text x="45" y="181" fill="currentColor" fontSize="11" textAnchor="middle">Input</text> <text x="163" y="181" fill="currentColor" fontSize="11" textAnchor="middle">Neuroni con stato</text> <text x="292" y="181" fill="currentColor" fontSize="11" textAnchor="middle">Output</text> <text x="241" y="15" fill="#d97706" fontSize="10" textAnchor="middle">Retroazione</text> </svg> <caption textAlign="center">Schema concettuale di un MLP esteso con stato interno e retroazione, non di un'architettura RumiAI già adottata.</caption> </box>

- **Memoria locale:** ogni neurone possiede uno stato che persiste nel tempo.
- **Retroazione:** l'attività può influenzare quella successiva, come nelle reti ricorrenti.
- **Connessioni adattive:** i collegamenti possono cambiare secondo una regola di apprendimento.
- **Specializzazione:** gruppi di neuroni possono rispondere a segnali o condizioni differenti.

Quest'ultimo passaggio apre anche alla ricerca sull'**Equilibrium Propagation**, dove l'apprendimento sfrutta le dinamiche di sistemi fisici. Esistono già dimostrazioni sperimentali su macchine Ising, pur con limitazioni rispetto a un'implementazione elettronica domestica. <Cite ref="turn868632search0"/>

---

5. Il confronto che farei realmente in un prossimo PoC

Confronterei quattro reti piccole, con un numero di parametri e risorse di calcolo il più possibile comparabile:

Candidato	Scopo
MLP tradizionale	Riferimento semplice, senza memoria ricorrente
MLP ricorrente	Misurare il vantaggio dello stato temporale
MLP ricorrente con connessioni adattive	Misurare l'utilità dell'apprendimento durante l'elaborazione
Rete con equilibrio dinamico	Misurare se la convergenza fisica può svolgere parte del calcolo

Userei prima un problema di previsione da osservazioni incomplete, con cambiamenti del processo nel tempo, misurando accuratezza, velocità di adattamento, robustezza al rumore e risorse necessarie.

I risultati potrebbero poi essere confrontati con un piccolo RWKV o un altro modello ricorrente digitale, sullo stesso compito.

La mia conclusione è che l'MLP non è un modello da superare prima di iniziare: è un possibile punto di partenza per costruire una rete fisica intelligente. Le architetture moderne ci suggeriscono quali proprietà aggiungere, mentre gli esperimenti devono stabilire se convenga realizzarle con circuiti fisici anziché soltanto con software.

In particolare, approfondirei **MLP ricorrente + apprendimento locale**, confrontandolo con DEQ e TTT-MLP. È il collegamento che vedo più promettente tra il vecchio perceptron multistrato e la direzione B che stiamo esplorando.

USER

confronta mlp con chatgpt

ASSISTANT

La differenza fondamentale è questa: **ChatGPT utilizza ancora il principio matematico del perceptron multistrato (MLP), ma lo inserisce in un sistema neurale molto più articolato, capace di elaborare relazioni fra informazioni distribuite nel contesto.**

Quindi non è corretto considerare il MLP una tecnologia superata e ChatGPT qualcosa di completamente diverso. Il Transformer, la famiglia di architetture da cui discendono i modelli GPT, contiene proprio reti feed-forward assimilabili a piccoli MLP. <Cite refs={["turn198655search37","turn198655search8"]}/>

---

1. Confronto diretto

Caratteristica	**MLP classico**	**ChatGPT / famiglia GPT**
Neuroni artificiali	Sì	Sì, in forme più articolate
Somme pesate e non linearità	Sì	Sì
Organizzazione	Livelli completamente connessi	Blocchi neurali profondi, con attenzione e trasformazioni feed-forward
Gestione delle relazioni	Incorporate nei pesi e nell'input	Calcolate dinamicamente anche tramite attention
Memoria del contesto	Occorre rappresentarla esplicitamente nell'input o nell'architettura	Il contesto viene elaborato con meccanismi dedicati
Apprendimento	Tipicamente backpropagation	Preaddestramento su larga scala e successive fasi di ottimizzazione
Generazione linguistica	Possibile con opportuni input/output, ma non intrinseca	Generazione autoregressiva
Complessità hardware	Relativamente semplice	Elevata
Realizzazione analogica	Più diretta	Possibile per sottosistemi, complessa per l'intero modello

Questa tabella confronta un MLP convenzionale con i principi pubblicamente documentati dei modelli GPT. **L'architettura interna esatta di GPT-6 non è pubblicamente descritta in dettaglio**: non possiamo quindi attribuirgli con certezza un numero di livelli, parametri o un particolare schema di attention. Già nel rapporto tecnico di GPT-4 OpenAI dichiarava di non pubblicare diversi dettagli architetturali. <Cite ref="turn198655search36"/>

---

2. Come cambia l'elaborazione

<box gap={3}> <grid columns={2} gap={3}> <grid-item> <box border radius="lg" padding={2} gap={2}> <text weight="medium" textAlign="center" size="sm">MLP classico</text> <svg viewBox="0 0 160 186" width="100%" xmlns="http://www.w3.org/2000/svg"> {#each [0,1,2] as a} {#each [0,1,2,3] as b} <line x1="22" y1={41+a*44} x2="81" y2={21+b*38} stroke="currentColor" strokeWidth="0.8" opacity="0.21"/> {/each} {/each} {#each [0,1,2,3] as a} {#each [0,1] as b} <line x1="81" y1={21+a*38} x2="140" y2={62+b*50} stroke="currentColor" strokeWidth="0.8"

opacity="0.21"/> {/each} {/each} {#each [0,1,2] as a} <circle cx="22" cy={41+a*44} r="8" fill="#64748b"/> {/each} {#each [0,1,2,3] as a} <circle cx="81" cy={21+a*38} r="9" fill="#2563eb"/> {/each} {#each [0,1] as a} <circle cx="140" cy={62+a*50} r="8" fill="#0d9488"/> {/each} <text x="80" y="179" textAnchor="middle" fill="currentColor" fontSize="10">Connessioni apprese</text> </svg> <caption textAlign="center">I dati attraversano livelli successivi di trasformazioni.</caption> </box> </grid-item> <grid-item> <box border radius="lg" padding={2} gap={2}> <text weight="medium" textAlign="center" size="sm">Transformer generativo</text> <box align="center" gap={1} padding={{top:2,bottom:2}}> <box background="surface-secondary" radius="md" padding={2} width="100%" align="center"> <text size="xs" weight="medium">Token e rappresentazioni</text> </box> <icon name="arrow-down" color="secondary"/> <box border radius="md" padding={2} width="100%" align="center"> <text size="xs" weight="medium">Self-attention</text> <text size="3xs" color="secondary">Relazioni contestuali</text> </box> <icon name="arrow-down" color="secondary"/> <box border radius="md" padding={2} width="100%" align="center"> <text size="xs" weight="medium">Feed-forward / MLP</text> <text size="3xs" color="secondary">Trasformazioni non lineari</text> </box> <icon name="refresh-cw" color="secondary"/> <text size="3xs" color="secondary">Blocchi ripetuti</text> </box> <caption textAlign="center">L'attenzione e gli MLP collaborano all'elaborazione del contesto.</caption> </box> </grid-item> </grid> </box>

Il calcolo dell'MLP

Un neurone artificiale convenzionale può essere descritto da:

$$ y=f\left(\sum_i w_i x_i+b\right) $$

Più neuroni e livelli successivi producono trasformazioni non lineari anche molto complesse.

Il calcolo del Transformer

Nel Transformer si aggiunge un meccanismo fondamentale:

$$ \operatorname{Attention}(Q,K,V) =\operatorname{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V $$

Questo consente di pesare dinamicamente le relazioni fra elementi del contesto. Il Transformer originale combina questo meccanismo con reti feed-forward applicate alle rappresentazioni. <Cite ref="turn198655search37"/>

La differenza è quindi soprattutto **come le informazioni vengono messe in relazione**.

Un MLP convenzionale riceve un vettore e lo trasforma attraverso connessioni apprese. Un Transformer può utilizzare il contenuto di un'intera sequenza per determinare quali informazioni considerare durante ogni elaborazione.

---

3. Perché ChatGPT è molto più capace di un normale MLP?

Non semplicemente perché possiede più neuroni.

Contano almeno tre fattori.

Il primo è l'architettura. I meccanismi di attention permettono di costruire rappresentazioni contestuali dinamiche, mentre un MLP tradizionale richiede che le relazioni rilevanti siano già codificate nell'input o apprese attraverso connessioni fisse.

Il secondo è l'addestramento. I modelli GPT vengono preaddestrati su grandi quantità di dati mediante obiettivi generativi e ulteriormente ottimizzati. Una parte importante delle loro capacità deriva da ciò che apprendono, non soltanto dalla struttura della rete.

Il terzo è la scala. Modelli più profondi e grandi, addestrati con molte risorse, possono apprendere rappresentazioni estremamente complesse.

È importante però ricordare che anche ChatGPT, durante la normale generazione, **non modifica continuamente tutti i propri pesi neurali** per apprendere da ogni messaggio. Il contesto cambia, mentre i parametri del modello rimangono generalmente fissi.

---

4. Cosa significa per la nostra ricerca RumiAI?

Per la strada B, il confronto suggerisce una domanda più interessante di «possiamo costruire un MLP grande quanto ChatGPT?».

La domanda è:

Possiamo ottenere funzioni equivalenti a memoria, elaborazione contestuale e apprendimento mediante dinamiche fisiche più semplici e meno costose?

Un MLP analogico potrebbe essere relativamente semplice da realizzare con somme di correnti, conduttanze regolabili e circuiti non lineari. Ma non basterebbe aumentare il numero di neuroni per ottenere automaticamente le capacità di ChatGPT.

Dovremmo studiare separatamente:

- **Elaborazione:** come una rete fisica effettua trasformazioni non lineari.
- **Memoria:** come conserva informazione su eventi precedenti.
- **Associazione:** come collega informazioni distanti o apparentemente diverse.
- **Apprendimento:** come cambia le proprie connessioni e conserva ciò che ha appreso.

I PoC 058 e 059 affrontano già alcuni aspetti della dinamica e della robustezza, ma non dimostrano ancora una capacità di apprendimento paragonabile a quella di un modello linguistico.

Per questo sceglierei un esperimento graduale: **MLP classico → MLP con stato ricorrente → MLP con apprendimento locale**, confrontando ogni variante con un piccolo Transformer su compiti identici di memoria, associazione e generalizzazione.

L'obiettivo non sarebbe dimostrare immediatamente che un MLP analogico supera ChatGPT, ma identificare quali capacità aggiuntive servono e se la fisica può fornirle in modo più efficiente.

La conclusione più importante: ChatGPT dimostra quanto sia potente combinare trasformazioni neurali relativamente semplici con meccanismi di contesto e un addestramento molto esteso. Questo lascia aperta una possibilità interessante per RumiAI: cercare un'altra organizzazione fisica delle stesse funzioni cognitive, senza dover imitare integralmente il Transformer.

Allegati e contenuti multimediali

Nessun allegato rilevato.